

ChessCanvas

Technical Architecture & Build Specification

A scriptable, deterministic chess-board rendering engine for programmatic video (Remotion) and canvas use — npm package design document.

Prepared for: implementation handoff to coding agent
Document type: engineering specification (build-ready)
Target runtime: Node.js / React / Remotion

Contents

1. Goals & Non-Goals
2. System Overview
3. Core Concepts — Piece IDs & Squares
4. Package & File Structure
5. Public Scripting API
6. Move Validation (chess.js integration)
7. Timeline & Deterministic Playback Model
8. Rendering Layer
9. Board Themes & Piece Sets
10. Check & Checkmate Animation Spec
11. Remotion Component Integration
12. NPM Packaging & Build Tooling
13. End-to-End Example

14. Implementation Roadmap (build order)

15. Open Decisions for the Agent

1. Goals & Non-Goals

ChessCanvas is an npm package that lets a developer script a chess game or strategy ("play this move, draw this arrow, highlight this square") and get back a deterministic, scrubbable visual that renders identically whether it's drawn live in a browser or frame-by-frame inside a Remotion video render.

In scope

- A scripting API: `.move()`, `.arrow()`, `.highlight()`, `.wait()`, `clear` helpers.
- Full legal-move validation backed by `chess.js`, mapped onto a fixed piece-ID system (e.g. BR1, WK).
- A deterministic timeline: any point in the game can be rendered from a normalized $t \in [0, 1]$ value with no dependency on previous frames.
- Canvas/SVG rendering of board, pieces, arrows, highlights, with swappable themes and piece sets.
- Check / checkmate animation, modeled on `chess.com`'s visual language, expressed in a frame-deterministic (not wall-clock CSS) way.
- A ready-to-use Remotion React component plus a standalone scripting engine usable outside React.

Out of scope (v1)

- Online multiplayer, PGN import from external sources, engine analysis (Stockfish) — can be layered on later via the same Engine module.
- Touch/drag interactivity for live play (this is a playback/render engine first; an interactive mode can reuse the same Engine + Timeline later).
- Sound design (can be added as a side-channel of timed events, not core to v1).

2. System Overview

The package has three independent layers. Keeping them independent is the most important architectural decision in this document — it is what makes Remotion playback deterministic.

Layer	Responsibility	Depends on
Logic (Engine + Board)	Legal move validation, piece-ID $\leftrightarrow$ square bookkeeping, game state	<code>chess.js</code>
Timeline	Records every scripted action as a time-ranged event; converts $t \rightarrow$ board state	Logic layer
Render	Pure function: $(\text{state}) \rightarrow \text{pixels/DOM}$. No internal clock, no side effects.	Timeline output

Critically: the Render layer never mutates state and never reads "now". It receives a single resolved state object for whatever t it was asked to draw, every time. This is what lets Remotion render frame 400 before frame 1 during parallel/segmented rendering and still get a correct picture.

Data flow



3. Core Concepts — Piece IDs & Squares

Every piece gets a permanent ID at game start and keeps it for the whole timeline, even after capture (captured pieces are marked alive: `false`, never deleted, so capture/undo/scrub-back animations remain possible).

ID pattern	Meaning	Example
W/B + piece letter + index	Color + type + disambiguation index (1-2 for officers, 1-8 for pawns)	BR1 = Black Rook #1 (queenside rook at game start)
K, Q (no index)	King and Queen are unique per side	WK = White King, BQ = Black Queen
N, B, R + 1/2	Knights, Bishops, Rooks — numbered queenside-to-kingside	WN1 = White Knight on b1, WN2 = White Knight on g1
P + 1..8	Pawns numbered a-file to h-file at game start	BP5 = Black pawn starting on e7

Squares use standard algebraic notation, a1–h8. The Board module owns a bidirectional map: `pieceId` → `square` and `square` → `pieceId`, kept in sync as the script is built (build time, not render time — see Section 7).

Promotion note

A pawn's ID (e.g. WP3) does not change on promotion — only its `type` field does, from that tick onward. The renderer looks up sprite by (`color`, `type`) resolved at the current tick, not by ID alone.

4. Package & File Structure

```
chesscanvas/  
  src/  
    core/  
      Engine.js          # wraps chess.js: legal-move validation, FEN/PGN bridging  
      Board.js           # pieceId <-> square map, capture/promotion bookkeeping  
      Timeline.js        # ordered, tick-ranged event list; t -> resolved state  
      events.js          # event type constants + factory helpers  
    render/  
      drawBoard.js       # squares, coordinates, board-theme fill  
      drawPieces.js      # piece sprites, position interpolation per piece  
      drawArrows.js      # arrow path geometry + arrowhead drawing  
      drawHighlights.js  # square highlight overlays  
      drawCheckOverlay.js # check / checkmate visual treatment (Section 10)  
      interpolate.js     # easing functions, square-to-pixel math  
    pieceSets/  
      cburnett/          # bundled default SVG set (own LICENSE file inside)  
      merida/  
      boardThemes.js     # named square-color palettes  
    api/  
      ChessCanvas.js     # public scripting class  
    remotion/  
      ChessCanvasComponent.jsx # React component consumed by Remotion compositions  
      useChessPlayback.js  # hook: frame -> t -> resolved state  
    index.js             # package entry: export { chesscanvas, ChessCanvas }  
    test/  
      engine.test.js  
      timeline.test.js  
      fixtures/          # known PGNs incl. scholar's mate, fool's mate, castling, en  
passant  
  assets/  
    LICENSE-cburnett.txt  
  package.json  
  tsup.config.ts (or rollup.config.js)  
  README.md
```

5. Public Scripting API

All scripting calls are **build-time** calls: they append validated events to a Timeline and return this for chaining. No drawing happens at call time.

Method	Signature	Behavior
move	move(pieceId, targetSquare, opts?)	Validates via Engine.validateMove(). On success: updates Board map, pushes a MOVE event with computed [startTick,endTick]. On failure: console.log('move is invalid', err); event NOT added; chain continues unaffected.
arrow	arrow(color, fromSquare, toSquare, opts?)	Pushes an ARROW event. No legality check (deliberately — lets users draw illegal/strategic arrows). opts: {persist: true false, duration}.

Method	Signature	Behavior
highlight	highlight(color, square, opts?)	Pushes a HIGHLIGHT event for one square. opts mirror arrow().
clearArrows	clearArrows()	Pushes a CLEAR_ARROWS marker event at current tick.
clearHighlights	clearHighlights()	Pushes a CLEAR_HIGHLIGHTS marker event.
wait	wait(units)	Advances the tick cursor without adding a visual event (a pause).
fen / pgn	fen(), pgn()	Debug accessors returning Engine's current logical position.
playback	playback(t)	t in [0,1]. Returns a resolved render-state object for that point in the whole script (see Section 7).

Example invalid-move handling

```
script.move("BR1", "a3");
// BR1 (black rook, a8) cannot legally reach a3 in one move from the
// current position -> Engine.validateMove returns { ok: false, reason: "..."}
// -> console.log("move is invalid", { piece: "BR1", to: "a3", reason: "..."} )
// -> timeline unchanged, script chain continues normally
```

6. Move Validation (chess.js integration)

Do not hand-roll chess rules. `chess.js` is the single source of truth for legality, check/checkmate/stalemate detection, castling rights, en passant and promotion. `Engine.js` is a thin adapter between piece IDs and `chess.js`'s square-based API.

Engine responsibilities

- Hold one `chess.js` instance representing logical game state.
- Resolve `pieceId` → `currentSquare` via `Board` before every move call.
- Call `chessJs.move({ from, to, promotion? });` `null` return = illegal.
- On legal move, read back special-move flags from `chess.js`'s move object (`flags` contains `k/q` for castling, `e` for en passant, `p` for promotion) and emit *additional* synthetic timeline events where needed (see below).
- After every move, call `chessJs.inCheck()` / `isCheckmate()` / `isStalemate()` and attach the result onto the MOVE event as `resultingState`.

Special-move handling

Case	What the user calls	What Engine emits internally
Castling	<code>move('WK','g1')</code>	MOVE for WK (e1→g1) + a synthetic MOVE for WR2 (h1→f1), same tick window, so both animate together
En passant	<code>move('WP5','d6')</code>	MOVE for WP5 + a synthetic CAPTURE flag marking the passed pawn as not-alive at the same tick, even though its square != target square
Promotion	<code>move('WP3','e8', {promotion:'Q'})</code>	MOVE event with a promotion field; Board updates WP3's logical type to Queen effective at endTick
Checkmate	any <code>move()</code> that ends the game	MOVE event's <code>resultingState.checkmate = true</code> ; Timeline appends a CHECKMATE marker event right after, consumed by <code>drawCheckOverlay</code> (Section 10)

7. Timeline & Deterministic Playback Model

This is the section that makes the package Remotion-safe. The core rule: **render state must be a pure function of t**, never of "what happened last frame."

Tick model

- Every event gets `[startTick, endTick]` in abstract timeline units, not seconds — this decouples the script from final video framerate/length.
- Default weight: a MOVE = 1 unit, an ARROW/HIGHLIGHT reveal = 0.25 units (near-instant), `wait(n) = n` units of dead time. All overridable per-call via `opts.duration`.
- `totalTicks` = sum of all event weights once the script is finalized (first call to `playback()` or an explicit `.finalize()`).

Resolving playback(t)

```
function playback(t) { // t in [0,1]
  const tick = t * totalTicks;

  // 1) Base state: replay every event whose endTick <= tick to completion.
  //    This is cheap because Board/Engine snapshots can be memoized at
  //    each event boundary the first time playback() walks past them.
  let state = getNearestMemoizedSnapshot(tick);

  // 2) In-flight event: the single event (if any) whose
  //    [startTick, endTick] window contains `tick`.
  const active = findEventContainingTick(tick);
  if (active && active.type === "MOVE") {
    const localT = (tick - active.startTick) / (active.endTick - active.startTick);
    state = interpolatePiecePosition(state, active, ease(localT));
  }

  // 3) Overlay events (arrows/highlights/check) are filtered to those
  //    whose window contains `tick`, independent of move interpolation.
  state.arrows = activeArrowsAt(tick);
  state.highlights = activeHighlightsAt(tick);
  state.checkState = activeCheckStateAt(tick); // none | check | checkmate

  return state; // pure, fully describes one frame
}
```

Memoizing a board snapshot at every event boundary (not every frame) keeps `playback(t)` $O(\log n)$ instead of $O(n)$ per call, which matters once a script has 40+ moves and Remotion is calling this once per output frame.

8. Rendering Layer

Render functions are pure: `draw*(ctx, state, theme)`. They never touch Engine, Board, or Timeline directly — only the resolved state object from `playback(t)`. Choose Canvas2D as the primary backend (best performance for Remotion's per-frame rasterization); keep an SVG/DOM fallback path optional for live-app scrubbing use cases outside video export.

Module	Input	Output
<code>drawBoard</code>	<code>boardTheme</code>	8x8 squares + optional coordinate labels
<code>drawPieces</code>	<code>state.pieces</code> (with interpolated x,y), <code>pieceSet</code>	piece sprites at current positions
<code>drawArrows</code>	<code>state.arrows</code>	arrow shaft + arrowhead per active arrow, alpha-faded if just appearing
<code>drawHighlights</code>	<code>state.highlights</code>	semi-transparent colored square fills, drawn under pieces
<code>drawCheckOverlay</code>	<code>state.checkState</code> , king square	pulsing/solid red treatment, see Section 10

Square ↔ pixel mapping

```
function squareToPixel(square, boardSize, orientation = "white") {
  const file = square.charCodeAt(0) - 97;    // 'a' -> 0
  const rank = parseInt(square[1], 10) - 1;  // '1' -> 0
  const cell = boardSize / 8;
  const col = orientation === "white" ? file : 7 - file;
  const row = orientation === "white" ? 7 - rank : rank;
  return { x: col * cell, y: row * cell, size: cell };
}
```

9. Board Themes & Piece Sets

Two independently swappable layers, both selectable through the top-level config object.

```
chesscanvas({
  boardTheme: "walnut",    // square color palette
  pieceSet: "cburnett",   // sprite set
});
```

Bundled board themes (boardThemes.js, plain color maps, no licensing concern)

- `classic` — cream / forest green (Lichess default look)
- `walnut` — warm brown wood tones
- `tournament` — high-contrast grey/white, broadcast style
- `blue` — chess.com-style cool blue/white

Bundled piece sets

- `cburnett` — the Lichess default SVG set, CC-BY-SA. Most recognizable "standard" look; ship as default.
- `merida` or `staunty` as a second bundled option for variety.
- Custom sets: `pieceSet` also accepts a user-supplied object keyed by piece code (`wK`, `wQ`, `wR`, `wN`, `wB`, `wP`, `bK`, ...) mapping to raw SVG/path data, so consumers can bring their own branded pieces without forking the package.

Licensing note

CC-BY-SA assets (`cburnett` and most Lichess sets) must keep their own license file (`assets/LICENSE-cburnett.txt`) separate from the package's own MIT license, with attribution preserved. Do not re-license bundled third-party SVGs under MIT.

10. Check & Checkmate Animation Spec

This section specifies the visual language to match chess.com's check/checkmate treatment, re-expressed as deterministic tick-based events instead of live CSS animation, since Remotion needs a frame-exact answer for any `t`, not an animation that "plays once and forgets."

10.1 What chess.com actually does (reference behavior)

- **On check:** the checked king's square gets a soft red radial gradient (red fading to transparent from the square's center) layered under the king sprite. It is a static highlight, not animated/pulsing, in chess.com's current UI.
- **On checkmate:** the checkmated king's square turns a flatter, fully opaque/solid red. The king piece itself is not removed or shaken — it stays on its square, but the board interaction freezes and a result panel/banner ("Checkmate — White/Black wins") fades in, usually with the winning side's avatar/name emphasized.
- Often the last (mating) move is shown with its standard last-move square highlight (two-square yellow highlight: from-square and to-square) still visible underneath/around the red king-square treatment.

10.2 ChessCanvas equivalent — event model

Both behaviors are modeled as overlay events on the king's square, driven by `resultingState` attached to the `MOVE` event that caused them (Section 6).

State	Trigger	Visual treatment	Persistence
check	<code>chessJs.inCheck() === true</code> after a move	Radial red gradient under the king sprite, center alpha ~0.55 fading to 0 at square edge. Drawn by <code>drawCheckOverlay</code> using <code>state.checkState === 'check'</code> .	Active from the move's <code>endTick</code> until the position is no longer in check (i.e. the next move that resolves it), or until end of script if check stands unresolved.
checkmate	<code>chessJs.isCheckmate() === true</code> after a move	Square fill becomes solid opaque red (alpha 1.0, no gradient). A <code>CHECKMATE</code> marker event is appended at the same tick; <code>drawCheckOverlay</code> switches treatment to solid fill once <code>state.checkState === 'checkmate'</code> .	Persists for the remainder of the timeline (t up to 1.0) — checkmate is terminal, nothing after it un-resolves the board.
last-move highlight	every successful <code>MOVE</code> event	Standard two-square highlight (from + to) in the theme's accent color, alpha ~0.4, drawn beneath pieces, same layer order as <code>user highlight()</code> calls.	Replaced by the next move's last-move highlight; remains under the check/checkmate overlay on the final move.

10.3 Pulse, expressed deterministically

If a subtle pulse is wanted on the *check* (not checkmate) state for extra production value, do not use a CSS `@keyframes` loop — Remotion needs the value at an exact frame, not "whatever phase a wall-clock animation happens to be in." Compute alpha as a deterministic function of tick instead:

```
// deterministic pulse: same result every time for the same tick,
// independent of when rendering started or whether frames are skipped
function checkPulseAlpha(tickSinceCheckStarted, periodTicks = 0.6) {
  const phase = (tickSinceCheckStarted % periodTicks) / periodTicks; // 0..1
  // sine-based pulse between 0.35 and 0.65 alpha
  return 0.5 + 0.15 * Math.sin(phase * Math.PI * 2);
}
```

Because `tickSinceCheckStarted` is derived purely from `tick - checkStartTick`, calling this with the same `t` always returns the same alpha — safe for out-of-order Remotion frame rendering.

10.4 Checkmate banner / result panel

Implement as its own overlay component, not baked into the canvas pieces layer, so it can be toggled off for users who only want the board (e.g. composing it themselves in a larger Remotion scene). Fade-in via tick-based alpha identical in spirit to the pulse function above (`alpha = clamp((tick - checkmateTick) / fadeInTicks, 0, 1)`), showing text such as "Checkmate — White wins" derived from `chessJs.turn()` at the mate position.

Expose this as `showResultBanner: true|false` in the top-level `chesscanvas()` config, default `true`.

11. Remotion Component Integration

The Remotion layer is a thin adapter: convert Remotion's frame counter into `t`, call `playback(t)`, `draw`.

```
// remotion/ChessCanvasComponent.jsx
import { useCurrentFrame, useVideoConfig } from "remotion";
import { useRef, useEffect } from "react";

export function ChessCanvasComponent({ timeline, boardTheme, pieceSet, showResultBanner }) {
  const frame = useCurrentFrame();
  const { durationInFrames, width, height } = useVideoConfig();
  const canvasRef = useRef(null);

  const t = durationInFrames <= 1 ? 0 : frame / (durationInFrames - 1);
  const state = timeline.playback(t); // pure, deterministic

  useEffect(() => {
    const ctx = canvasRef.current.getContext("2d");
    ctx.clearRect(0, 0, width, height);
    drawBoard(ctx, boardTheme);
    drawHighlights(ctx, state.highlights);
    drawArrows(ctx, state.arrows);
    drawPieces(ctx, state.pieces, pieceSet);
    drawCheckOverlay(ctx, state.checkState, state.kingSquare);
  }, [t]); // re-draw whenever the resolved frame changes

  return (
    <>
      <canvas ref={canvasRef} width={width} height={height} />
      {showResultBanner && state.checkState === "checkmate" && (
        <ResultBanner winner={state.winner} alpha={state.bannerAlpha} />
      )}
    </>
  );
}
```

The package's entry point wraps script-building and component creation together:

```
// index.js
export function chesscanvas(config = {}) {
  const engine = new Engine();
  const board = new Board(engine);
  const timeline = new Timeline(engine, board);
  const script = new ChessCanvas(engine, board, timeline);

  const Component = (props) => (
    <ChessCanvasComponent
      timeline={timeline}
      boardTheme={config.boardTheme ?? "classic"}
      pieceSet={config.pieceSet ?? "cburnett"}
      showResultBanner={config.showResultBanner ?? true}
      {...props}
    />
  );

  return { script, Component, timeline };
}
```

Note: durationInFrames for the Remotion <Composition> should be derived from the script's totalTicks times a chosen ticks-per-second rate (e.g. 1 tick = 45 frames at 30fps $\approx$ 1.5s per move) — expose a helper

`timeline.suggestedDurationInFrames(fps)` so the consumer doesn't have to hand-compute it.

12. NPM Packaging & Build Tooling

Concern	Decision
Peer dependencies	react, react-dom, remotion — not bundled; consumer's project supplies versions
Hard dependency	chess.js (pinned to a tested minor version)
Bundler	tsup (esbuild-based) producing both ESM and CJS outputs in dist/
Exports map	'.' -> core API (works without React); './remotion' -> React/Remotion-specific exports, so non-React consumers of the scripting/validation engine aren't forced to pull in React
TypeScript	Recommend authoring in TS even though the user's brief is plain JS — emit .d.ts for editor autocomplete on the scripting API; ship as a build step, source can still read like JS
Testing	vitest or jest; fixture PGNs for castling, en passant, promotion, scholar's mate, fool's mate, stalemate — assert Timeline event sequences, not just chess.js correctness
Versioning	semver from day one; treat the piece-ID scheme and event-shape as the public contract — breaking changes to either are major-version bumps

13. End-to-End Example

```
import { chesscanvas } from "chesscanvas";
import { Composition } from "remotion";

const { script, Component, timeline } = chesscanvas({
  boardTheme: "walnut",
  pieceSet: "cburnett",
});

// Scholar's mate, annotated
script
  .move("WP5", "e4")
  .move("BP5", "e5")
  .move("WB2", "c4")
  .move("BN1", "c6")
  .move("WQ", "h5")
  .highlight("yellow", "f7")
  .arrow("red", "h5", "f7")
  .move("BN2", "f6")           // black misses the threat
  .move("WQ", "f7");           // checkmate

export const ScholarsMate = () => (
  <Composition
    id="scholars-mate"
    component={Component}
    durationInFrames={timeline.suggestedDurationInFrames(30)}
    fps={30}
    width={1080}
    height={1080}
  />
);
```

If `move("BN2", "f6")` had instead been an illegal square, the console would log `"move is invalid"` with the reason, that single call would be a no-op on the timeline, and every subsequent chained call would still execute normally.

14. Implementation Roadmap (build order)

Phase	Deliverable	Done when
1	Engine.js + Board.js, no rendering	chess.js wrapped; piece-ID move validation passes fixture tests incl. castling/en passant/promotion
2	Timeline.js event recording + tick math	script.move()/arrow()/highlight() correctly build an ordered, tick-ranged event list; invalid moves are no-ops
3	Static render at t=1 (final position)	drawBoard + drawPieces show correct final board for a known PGN, no animation yet
4	autoplay(t) + interpolate.js	scrubbing t from 0 to 1 smoothly animates each move in sequence; memoized snapshots keep it fast
5	drawArrows.js + drawHighlights.js	arrows/highlights appear/disappear at correct ticks, drawn under pieces
6	drawCheckOverlay.js (Section 10)	check radial + checkmate solid-fill + result banner all keyed off resultingState, frame-deterministic
7	ChessCanvasComponent.jsx + Remotion wiring	a real Remotion composition renders a full game to mp4 with correct timing
8	Theming: board themes + 2 piece sets + custom pieceSet override	swapping boardTheme/pieceSet visibly changes output; custom SVG object works
9	Packaging: exports map, tsup build, peer deps, README, license files	`npm pack` produces a clean installable tarball; fresh Remotion project can `npm install` and import successfully

15. Open Decisions for the Agent

These were intentionally left flexible in this spec; pick sensible defaults and document the choice in the README rather than blocking on them.

- Default move duration in ticks/seconds — 1.0–1.5s per move is a reasonable starting default.
- Whether last-move highlight color is theme-controlled or fixed across all themes.
- Whether arrows support curved/knight-shaped paths (chess.com bends knight-move arrows) or only straight lines in v1 — recommend straight-only for v1, revisit if needed.
- Exact easing curve for piece movement (ease-in-out cubic is a safe, natural-looking default).
- Whether to expose an imperative seek (t) convenience alongside p layback (t) for non-Remotion/live-app consumers.

End of specification. Hand this document plus the package skeleton in Section 4 directly to the implementation agent as the build brief.

ChessCanvas

Technical Architecture & Build Specification

A scriptable, deterministic chess-board rendering engine for programmatic video (Remotion) and canvas use — npm package design document.

Prepared for: implementation handoff to coding agent
Document type: engineering specification (build-ready)
Target runtime: Node.js / React / Remotion

Contents

1. Goals & Non-Goals
2. System Overview
3. Core Concepts — Piece IDs & Squares
4. Package & File Structure
5. Public Scripting API
6. Move Validation (chess.js integration)
7. Timeline & Deterministic Playback Model
8. Rendering Layer
9. Board Themes & Piece Sets
10. Check & Checkmate Animation Spec
11. Remotion Component Integration
12. NPM Packaging & Build Tooling
13. End-to-End Example

14. Implementation Roadmap (build order)
15. Open Decisions for the Agent

1. Goals & Non-Goals

ChessCanvas is an npm package that lets a developer script a chess game or strategy ("play this move, draw this arrow, highlight this square") and get back a deterministic, scrubbable visual that renders identically whether it's drawn live in a browser or frame-by-frame inside a Remotion video render.

In scope

- A scripting API: `.move()`, `.arrow()`, `.highlight()`, `.wait()`, `clear` helpers.
- Full legal-move validation backed by `chess.js`, mapped onto a fixed piece-ID system (e.g. BR1, WK).
- A deterministic timeline: any point in the game can be rendered from a normalized $t \in [0, 1]$ value with no dependency on previous frames.
- Canvas/SVG rendering of board, pieces, arrows, highlights, with swappable themes and piece sets.
- Check / checkmate animation, modeled on `chess.com`'s visual language, expressed in a frame-deterministic (not wall-clock CSS) way.
- A ready-to-use Remotion React component plus a standalone scripting engine usable outside React.

Out of scope (v1)

- Online multiplayer, PGN import from external sources, engine analysis (Stockfish) — can be layered on later via the same Engine module.
- Touch/drag interactivity for live play (this is a playback/render engine first; an interactive mode can reuse the same Engine + Timeline later).
- Sound design (can be added as a side-channel of timed events, not core to v1).

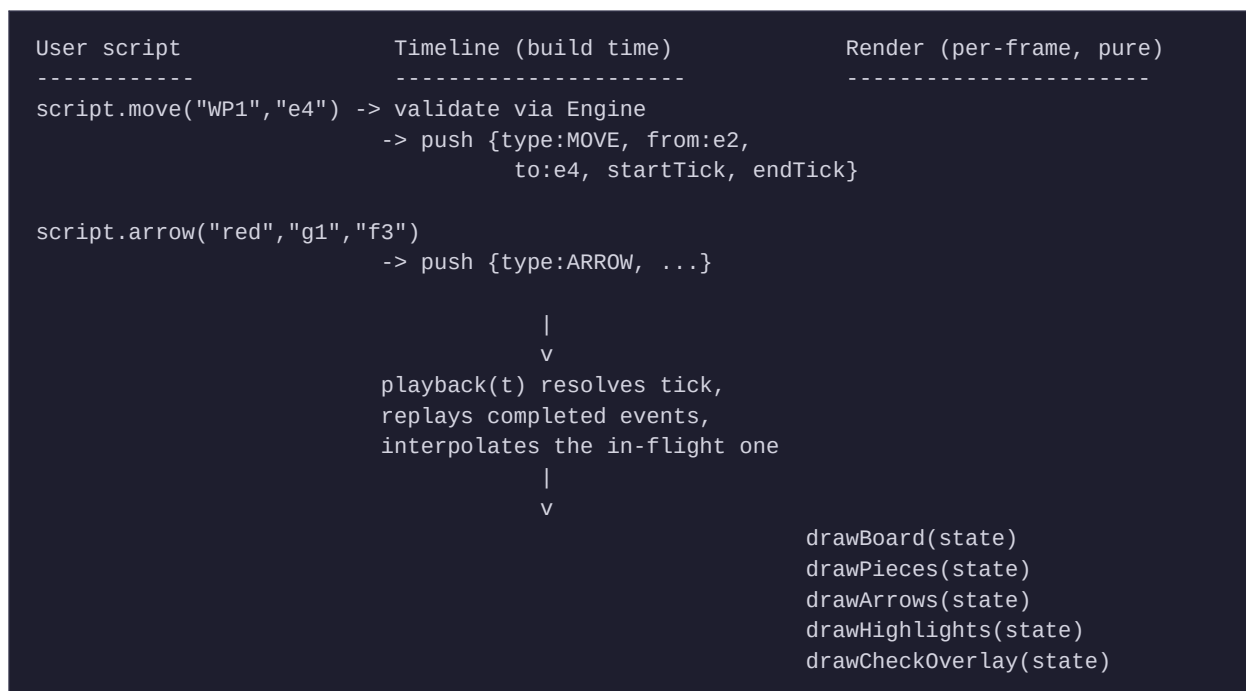
2. System Overview

The package has three independent layers. Keeping them independent is the most important architectural decision in this document — it is what makes Remotion playback deterministic.

Layer	Responsibility	Depends on
Logic (Engine + Board)	Legal move validation, piece-ID $\leftrightarrow$ square bookkeeping, game state	<code>chess.js</code>
Timeline	Records every scripted action as a time-ranged event; converts $t \rightarrow$ board state	Logic layer
Render	Pure function: $(\text{state}) \rightarrow \text{pixels/DOM}$. No internal clock, no side effects.	Timeline output

Critically: the Render layer never mutates state and never reads "now". It receives a single resolved state object for whatever t it was asked to draw, every time. This is what lets Remotion render frame 400 before frame 1 during parallel/segmented rendering and still get a correct picture.

Data flow



3. Core Concepts — Piece IDs & Squares

Every piece gets a permanent ID at game start and keeps it for the whole timeline, even after capture (captured pieces are marked alive: `false`, never deleted, so capture/undo/scrub-back animations remain possible).

ID pattern	Meaning	Example
W/B + piece letter + index	Color + type + disambiguation index (1-2 for officers, 1-8 for pawns)	BR1 = Black Rook #1 (queenside rook at game start)
K, Q (no index)	King and Queen are unique per side	WK = White King, BQ = Black Queen
N, B, R + 1/2	Knights, Bishops, Rooks — numbered queenside-to-kingside	WN1 = White Knight on b1, WN2 = White Knight on g1
P + 1..8	Pawns numbered a-file to h-file at game start	BP5 = Black pawn starting on e7

Squares use standard algebraic notation, a1–h8. The Board module owns a bidirectional map: `pieceId` → `square` and `square` → `pieceId`, kept in sync as the script is built (build time, not render time — see Section 7).

Promotion note

A pawn's ID (e.g. WP3) does not change on promotion — only its `type` field does, from that tick onward. The renderer looks up sprite by (`color`, `type`) resolved at the current tick, not by ID alone.

4. Package & File Structure

```
chesscanvas/  
  src/  
    core/  
      Engine.js      # wraps chess.js: legal-move validation, FEN/PGN bridging  
      Board.js       # pieceId <-> square map, capture/promotion bookkeeping  
      Timeline.js    # ordered, tick-ranged event list; t -> resolved state  
      events.js      # event type constants + factory helpers  
    render/  
      drawBoard.js   # squares, coordinates, board-theme fill  
      drawPieces.js  # piece sprites, position interpolation per piece  
      drawArrows.js  # arrow path geometry + arrowhead drawing  
      drawHighlights.js # square highlight overlays  
      drawCheckOverlay.js # check / checkmate visual treatment (Section 10)  
      interpolate.js # easing functions, square-to-pixel math  
    pieceSets/  
      cburnett/      # bundled default SVG set (own LICENSE file inside)  
      merida/  
      boardThemes.js # named square-color palettes  
    api/  
      ChessCanvas.js # public scripting class  
    remotion/  
      ChessCanvasComponent.jsx # React component consumed by Remotion compositions  
      useChessPlayback.js      # hook: frame -> t -> resolved state  
    index.js              # package entry: export { chesscanvas, ChessCanvas }  
    test/  
      engine.test.js  
      timeline.test.js  
      fixtures/          # known PGNs incl. scholar's mate, fool's mate, castling, en  
passant  
  assets/  
    LICENSE-cburnett.txt  
  package.json  
  tsup.config.ts (or rollup.config.js)  
  README.md
```

5. Public Scripting API

All scripting calls are **build-time** calls: they append validated events to a Timeline and return this for chaining. No drawing happens at call time.

Method	Signature	Behavior
move	move(pieceId, targetSquare, opts?)	Validates via Engine.validateMove(). On success: updates Board map, pushes a MOVE event with computed [startTick,endTick]. On failure: console.log('move is invalid', err); event NOT added; chain continues unaffected.
arrow	arrow(color, fromSquare, toSquare, opts?)	Pushes an ARROW event. No legality check (deliberately — lets users draw illegal/strategic arrows). opts: {persist: true false, duration}.

Method	Signature	Behavior
highlight	highlight(color, square, opts?)	Pushes a HIGHLIGHT event for one square. opts mirror arrow().
clearArrows	clearArrows()	Pushes a CLEAR_ARROWS marker event at current tick.
clearHighlights	clearHighlights()	Pushes a CLEAR_HIGHLIGHTS marker event.
wait	wait(units)	Advances the tick cursor without adding a visual event (a pause).
fen / pgn	fen(), pgn()	Debug accessors returning Engine's current logical position.
playback	playback(t)	t in [0,1]. Returns a resolved render-state object for that point in the whole script (see Section 7).

Example invalid-move handling

```
script.move("BR1", "a3");
// BR1 (black rook, a8) cannot legally reach a3 in one move from the
// current position -> Engine.validateMove returns { ok: false, reason: "..."}
// -> console.log("move is invalid", { piece: "BR1", to: "a3", reason: "..."} )
// -> timeline unchanged, script chain continues normally
```

6. Move Validation (chess.js integration)

Do not hand-roll chess rules. `chess.js` is the single source of truth for legality, check/checkmate/stalemate detection, castling rights, en passant and promotion. `Engine.js` is a thin adapter between piece IDs and `chess.js`'s square-based API.

Engine responsibilities

- Hold one `chess.js` instance representing logical game state.
- Resolve `pieceId` → `currentSquare` via `Board` before every move call.
- Call `chessJs.move({ from, to, promotion? });` `null` return = illegal.
- On legal move, read back special-move flags from `chess.js`'s move object (`flags` contains `k/q` for castling, `e` for en passant, `p` for promotion) and emit *additional* synthetic timeline events where needed (see below).
- After every move, call `chessJs.inCheck()` / `isCheckmate()` / `isStalemate()` and attach the result onto the MOVE event as `resultingState`.

Special-move handling

Case	What the user calls	What Engine emits internally
Castling	<code>move('WK','g1')</code>	MOVE for WK (e1→g1) + a synthetic MOVE for WR2 (h1→f1), same tick window, so both animate together
En passant	<code>move('WP5','d6')</code>	MOVE for WP5 + a synthetic CAPTURE flag marking the passed pawn as not-alive at the same tick, even though its square != target square
Promotion	<code>move('WP3','e8', {promotion:'Q'})</code>	MOVE event with a promotion field; Board updates WP3's logical type to Queen effective at endTick
Checkmate	any <code>move()</code> that ends the game	MOVE event's <code>resultingState.checkmate = true</code> ; Timeline appends a CHECKMATE marker event right after, consumed by <code>drawCheckOverlay</code> (Section 10)

7. Timeline & Deterministic Playback Model

This is the section that makes the package Remotion-safe. The core rule: **render state must be a pure function of t**, never of "what happened last frame."

Tick model

- Every event gets `[startTick, endTick]` in abstract timeline units, not seconds — this decouples the script from final video framerate/length.
- Default weight: a MOVE = 1 unit, an ARROW/HIGHLIGHT reveal = 0.25 units (near-instant), `wait(n) = n` units of dead time. All overridable per-call via `opts.duration`.
- `totalTicks` = sum of all event weights once the script is finalized (first call to `playback()` or an explicit `.finalize()`).

Resolving playback(t)

```
function playback(t) { // t in [0,1]
  const tick = t * totalTicks;

  // 1) Base state: replay every event whose endTick <= tick to completion.
  //    This is cheap because Board/Engine snapshots can be memoized at
  //    each event boundary the first time playback() walks past them.
  let state = getNearestMemoizedSnapshot(tick);

  // 2) In-flight event: the single event (if any) whose
  //    [startTick, endTick] window contains `tick`.
  const active = findEventContainingTick(tick);
  if (active && active.type === "MOVE") {
    const localT = (tick - active.startTick) / (active.endTick - active.startTick);
    state = interpolatePiecePosition(state, active, ease(localT));
  }

  // 3) Overlay events (arrows/highlights/check) are filtered to those
  //    whose window contains `tick`, independent of move interpolation.
  state.arrows      = activeArrowsAt(tick);
  state.highlights  = activeHighlightsAt(tick);
  state.checkState  = activeCheckStateAt(tick); // none | check | checkmate

  return state; // pure, fully describes one frame
}
```

Memoizing a board snapshot at every event boundary (not every frame) keeps `playback(t)` $O(\log n)$ instead of $O(n)$ per call, which matters once a script has 40+ moves and Remotion is calling this once per output frame.

8. Rendering Layer

Render functions are pure: `draw*(ctx, state, theme)`. They never touch Engine, Board, or Timeline directly — only the resolved state object from `playback(t)`. Choose Canvas2D as the primary backend (best performance for Remotion's per-frame rasterization); keep an SVG/DOM fallback path optional for live-app scrubbing use cases outside video export.

Module	Input	Output
<code>drawBoard</code>	<code>boardTheme</code>	8x8 squares + optional coordinate labels
<code>drawPieces</code>	<code>state.pieces</code> (with interpolated x,y), <code>pieceSet</code>	piece sprites at current positions
<code>drawArrows</code>	<code>state.arrows</code>	arrow shaft + arrowhead per active arrow, alpha-faded if just appearing
<code>drawHighlights</code>	<code>state.highlights</code>	semi-transparent colored square fills, drawn under pieces
<code>drawCheckOverlay</code>	<code>state.checkState</code> , king square	pulsing/solid red treatment, see Section 10

Square ↔ pixel mapping

```
function squareToPixel(square, boardSize, orientation = "white") {
  const file = square.charCodeAt(0) - 97;    // 'a' -> 0
  const rank = parseInt(square[1], 10) - 1;   // '1' -> 0
  const cell = boardSize / 8;
  const col = orientation === "white" ? file : 7 - file;
  const row = orientation === "white" ? 7 - rank : rank;
  return { x: col * cell, y: row * cell, size: cell };
}
```

9. Board Themes & Piece Sets

Two independently swappable layers, both selectable through the top-level config object.

```
chesscanvas({
  boardTheme: "walnut",    // square color palette
  pieceSet: "cburnett",    // sprite set
});
```

Bundled board themes (boardThemes.js, plain color maps, no licensing concern)

- `classic` — cream / forest green (Lichess default look)
- `walnut` — warm brown wood tones
- `tournament` — high-contrast grey/white, broadcast style
- `blue` — chess.com-style cool blue/white

Bundled piece sets

- `cburnett` — the Lichess default SVG set, CC-BY-SA. Most recognizable "standard" look; ship as default.
- `merida` or `staunty` as a second bundled option for variety.
- Custom sets: `pieceSet` also accepts a user-supplied object keyed by piece code (`wK`, `wQ`, `wR`, `wN`, `wB`, `wP`, `bK`, ...) mapping to raw SVG/path data, so consumers can bring their own branded pieces without forking the package.

Licensing note

CC-BY-SA assets (`cburnett` and most Lichess sets) must keep their own license file (`assets/LICENSE-cburnett.txt`) separate from the package's own MIT license, with attribution preserved. Do not re-license bundled third-party SVGs under MIT.

10. Check & Checkmate Animation Spec

This section specifies the visual language to match chess.com's check/checkmate treatment, re-expressed as deterministic tick-based events instead of live CSS animation, since Remotion needs a frame-exact answer for any `t`, not an animation that "plays once and forgets."

10.1 What chess.com actually does (reference behavior)

- **On check:** the checked king's square gets a soft red radial gradient (red fading to transparent from the square's center) layered under the king sprite. It is a static highlight, not animated/pulsing, in chess.com's current UI.
- **On checkmate:** the checkmated king's square turns a flatter, fully opaque/solid red. The king piece itself is not removed or shaken — it stays on its square, but the board interaction freezes and a result panel/banner ("Checkmate — White/Black wins") fades in, usually with the winning side's avatar/name emphasized.
- Often the last (mating) move is shown with its standard last-move square highlight (two-square yellow highlight: from-square and to-square) still visible underneath/around the red king-square treatment.

10.2 ChessCanvas equivalent — event model

Both behaviors are modeled as overlay events on the king's square, driven by `resultingState` attached to the `MOVE` event that caused them (Section 6).

State	Trigger	Visual treatment	Persistence
check	<code>chessJs.inCheck() === true</code> after a move	Radial red gradient under the king sprite, center alpha ~0.55 fading to 0 at square edge. Drawn by <code>drawCheckOverlay</code> using <code>state.checkState === 'check'</code> .	Active from the move's <code>endTick</code> until the position is no longer in check (i.e. the next move that resolves it), or until end of script if check stands unresolved.
checkmate	<code>chessJs.isCheckmate() === true</code> after a move	Square fill becomes solid opaque red (alpha 1.0, no gradient). A <code>CHECKMATE</code> marker event is appended at the same tick; <code>drawCheckOverlay</code> switches treatment to solid fill once <code>state.checkState === 'checkmate'</code> .	Persists for the remainder of the timeline (t up to 1.0) — checkmate is terminal, nothing after it un-resolves the board.
last-move highlight	every successful <code>MOVE</code> event	Standard two-square highlight (from + to) in the theme's accent color, alpha ~0.4, drawn beneath pieces, same layer order as <code>user highlight()</code> calls.	Replaced by the next move's last-move highlight; remains under the check/checkmate overlay on the final move.

10.3 Pulse, expressed deterministically

If a subtle pulse is wanted on the *check* (not checkmate) state for extra production value, do not use a CSS `@keyframes` loop — Remotion needs the value at an exact frame, not "whatever phase a wall-clock animation happens to be in." Compute alpha as a deterministic function of tick instead:

```
// deterministic pulse: same result every time for the same tick,
// independent of when rendering started or whether frames are skipped
function checkPulseAlpha(tickSinceCheckStarted, periodTicks = 0.6) {
  const phase = (tickSinceCheckStarted % periodTicks) / periodTicks; // 0..1
  // sine-based pulse between 0.35 and 0.65 alpha
  return 0.5 + 0.15 * Math.sin(phase * Math.PI * 2);
}
```

Because `tickSinceCheckStarted` is derived purely from `tick - checkStartTick`, calling this with the same `t` always returns the same alpha — safe for out-of-order Remotion frame rendering.

10.4 Checkmate banner / result panel

Implement as its own overlay component, not baked into the canvas pieces layer, so it can be toggled off for users who only want the board (e.g. composing it themselves in a larger Remotion scene). Fade-in via tick-based alpha identical in spirit to the pulse function above (`alpha = clamp((tick - checkmateTick) / fadeInTicks, 0, 1)`), showing text such as "Checkmate — White wins" derived from `chessJs.turn()` at the mate position.

Expose this as `showResultBanner: true|false` in the top-level `chesscanvas()` config, default `true`.

11. Remotion Component Integration

The Remotion layer is a thin adapter: convert Remotion's frame counter into `t`, call `playback(t)`, `draw`.

```
// remotion/ChessCanvasComponent.jsx
import { useCurrentFrame, useVideoConfig } from "remotion";
import { useRef, useEffect } from "react";

export function ChessCanvasComponent({ timeline, boardTheme, pieceSet, showResultBanner }) {
  const frame = useCurrentFrame();
  const { durationInFrames, width, height } = useVideoConfig();
  const canvasRef = useRef(null);

  const t = durationInFrames <= 1 ? 0 : frame / (durationInFrames - 1);
  const state = timeline.playback(t); // pure, deterministic

  useEffect(() => {
    const ctx = canvasRef.current.getContext("2d");
    ctx.clearRect(0, 0, width, height);
    drawBoard(ctx, boardTheme);
    drawHighlights(ctx, state.highlights);
    drawArrows(ctx, state.arrows);
    drawPieces(ctx, state.pieces, pieceSet);
    drawCheckOverlay(ctx, state.checkState, state.kingSquare);
  }, [t]); // re-draw whenever the resolved frame changes

  return (
    <>
      <canvas ref={canvasRef} width={width} height={height} />
      {showResultBanner && state.checkState === "checkmate" && (
        <ResultBanner winner={state.winner} alpha={state.bannerAlpha} />
      )}
    </>
  );
}
```

The package's entry point wraps script-building and component creation together:

```
// index.js
export function chesscanvas(config = {}) {
  const engine = new Engine();
  const board = new Board(engine);
  const timeline = new Timeline(engine, board);
  const script = new ChessCanvas(engine, board, timeline);

  const Component = (props) => (
    <ChessCanvasComponent
      timeline={timeline}
      boardTheme={config.boardTheme ?? "classic"}
      pieceSet={config.pieceSet ?? "cburnett"}
      showResultBanner={config.showResultBanner ?? true}
      {...props}
    />
  );

  return { script, Component, timeline };
}
```

Note: durationInFrames for the Remotion <Composition> should be derived from the script's totalTicks times a chosen ticks-per-second rate (e.g. 1 tick = 45 frames at 30fps $\approx$ 1.5s per move) — expose a helper

`timeline.suggestedDurationInFrames(fps)` so the consumer doesn't have to hand-compute it.

12. NPM Packaging & Build Tooling

Concern	Decision
Peer dependencies	react, react-dom, remotion — not bundled; consumer's project supplies versions
Hard dependency	chess.js (pinned to a tested minor version)
Bundler	tsup (esbuild-based) producing both ESM and CJS outputs in dist/
Exports map	'.' -> core API (works without React); './remotion' -> React/Remotion-specific exports, so non-React consumers of the scripting/validation engine aren't forced to pull in React
TypeScript	Recommend authoring in TS even though the user's brief is plain JS — emit .d.ts for editor autocomplete on the scripting API; ship as a build step, source can still read like JS
Testing	vitest or jest; fixture PGNs for castling, en passant, promotion, scholar's mate, fool's mate, stalemate — assert Timeline event sequences, not just chess.js correctness
Versioning	semver from day one; treat the piece-ID scheme and event-shape as the public contract — breaking changes to either are major-version bumps

13. End-to-End Example

```
import { chesscanvas } from "chesscanvas";
import { Composition } from "remotion";

const { script, Component, timeline } = chesscanvas({
  boardTheme: "walnut",
  pieceSet: "cburnett",
});

// Scholar's mate, annotated
script
  .move("WP5", "e4")
  .move("BP5", "e5")
  .move("WB2", "c4")
  .move("BN1", "c6")
  .move("WQ", "h5")
  .highlight("yellow", "f7")
  .arrow("red", "h5", "f7")
  .move("BN2", "f6")           // black misses the threat
  .move("WQ", "f7");          // checkmate

export const ScholarsMate = () => (
  <Composition
    id="scholars-mate"
    component={Component}
    durationInFrames={timeline.suggestedDurationInFrames(30)}
    fps={30}
    width={1080}
    height={1080}
  />
);
```

If `move("BN2", "f6")` had instead been an illegal square, the console would log `"move is invalid"` with the reason, that single call would be a no-op on the timeline, and every subsequent chained call would still execute normally.

14. Implementation Roadmap (build order)

Phase	Deliverable	Done when
1	Engine.js + Board.js, no rendering	chess.js wrapped; piece-ID move validation passes fixture tests incl. castling/en passant/promotion
2	Timeline.js event recording + tick math	script.move()/arrow()/highlight() correctly build an ordered, tick-ranged event list; invalid moves are no-ops
3	Static render at t=1 (final position)	drawBoard + drawPieces show correct final board for a known PGN, no animation yet
4	playback(t) + interpolate.js	scrubbing t from 0 to 1 smoothly animates each move in sequence; memoized snapshots keep it fast
5	drawArrows.js + drawHighlights.js	arrows/highlights appear/disappear at correct ticks, drawn under pieces
6	drawCheckOverlay.js (Section 10)	check radial + checkmate solid-fill + result banner all keyed off resultingState, frame-deterministic
7	ChessCanvasComponent.jsx + Remotion wiring	a real Remotion composition renders a full game to mp4 with correct timing
8	Theming: board themes + 2 piece sets + custom pieceSet override	swapping boardTheme/pieceSet visibly changes output; custom SVG object works
9	Packaging: exports map, tsup build, peer deps, README, license files	`npm pack` produces a clean installable tarball; fresh Remotion project can `npm install` and import successfully

15. Open Decisions for the Agent

These were intentionally left flexible in this spec; pick sensible defaults and document the choice in the README rather than blocking on them.

- Default move duration in ticks/seconds — 1.0–1.5s per move is a reasonable starting default.
- Whether last-move highlight color is theme-controlled or fixed across all themes.
- Whether arrows support curved/knight-shaped paths (chess.com bends knight-move arrows) or only straight lines in v1 — recommend straight-only for v1, revisit if needed.
- Exact easing curve for piece movement (ease-in-out cubic is a safe, natural-looking default).
- Whether to expose an imperative `seek(t)` convenience alongside `playback(t)` for non-Remotion/live-app consumers.

End of specification. Hand this document plus the package skeleton in Section 4 directly to the implementation agent as the build brief.